Parallel (using P-threads) 9-Point 2D Stencil Kernel

In this assignment you will implement a parallel version of your 2d stencil from the prior assignment (at the end of this assignment).

Your parallelization will be row decomposition (each thread will get he's part of the rows of the 2d metal plate).

You'll be adding these programs to your existing (working) serial version:

pth-stencil-2d.c // parallel pthread-based implementation

stencil_2d_tester.py // driver to iteratively test different sizes and number of threads to make sure that the parallel version ALWAYS gives you the exact same answers for final and stack as the serial version

bench_stencil2d_pthreads.py // driver program to run and collect timing information necessary to produce, timing, efficiency, speedup, and iso-efficiency plots as a function of SIZE, NUMTHREADS, and NUM_ITERATIONS.

See the details below for exact interfaces etc.

The parallel version show have this EXACT interface:

```
(base) wjones@CCU022633 source % ./pth-stencil-2d
Usage: ./pth-stencil-2d -n <iters> -I <in.raw> -o <out.raw> [-s <stack.raw>]
-t <threads>
  -n ITERS      Number of iterations (time steps)
  -I FILE       Input grid (.raw) header+data (int rows, int cols, doubles)
  -o FILE       Output final grid (.raw) header+data
  -s FILE       (optional) Raw stack output file (writes header + all frames)
  -t NTHREADS   Number of pthreads (>=1)
  -h            Help
(base) wjones@CCU022633 source %
```

Example of sequential execution and parallel execution and showing that the achieved results are the same:

```
(base) wjones@CCU022633 source % ./make-2d 4000 2000 initial.dat
(base) wjones@CCU022633 source % ./stencil-2d 500 initial.dat final.dat all.dat
(base) wjones@CCU022633 source % ls -l *.dat
-rw-r--r--  1 wjones  staff     40080008 Oct 20 09:04 all.100x100x500.dat
-rw-r--r--  1 wjones  staff    400080008 Oct 20 09:18 all.100x100x5000.dat
-rw-r--r--  1 wjones  staff   4000080008 Oct 20 09:19 all.100x100x50000.dat
-rw-r--r--  1 wjones  staff  32064000008 Oct 27 09:18 all.dat
-rw-r--r--  1 wjones  staff     64000008 Oct 27 09:18 final.dat
-rw-r--r--  1 wjones  staff     64000008 Oct 27 09:17 initial.dat
(base) wjones@CCU022633 source % ./pth-stencil-2d
Usage: ./pth-stencil-2d -n <iters> -I <in.raw> -o <out.raw> [-s <stack.raw>] -t <threads>
  -n ITERS      Number of iterations (time steps)
  -I FILE       Input grid (.raw) header+data (int rows, int cols, doubles)
  -o FILE       Output final grid (.raw) header+data
  -s FILE       (optional) Raw stack output file (writes header + all frames)
  -t NTHREADS   Number of pthreads (>=1)
  -h            Help
(base) wjones@CCU022633 source % ./pth-stencil-2d -n 500 -I ./initial.dat -o ./final.dat-pth
-s all.dat.pth -t 8
(base) wjones@CCU022633 source % diff final.dat final.dat-pth
(base) wjones@CCU022633 source % diff all.dat all.dat.pth
(base) wjones@CCU022633 source %
```

You should have a tester program that will make sure that if we run lots of different sizes and numbers of pthreads, that we always get the correct answers. Think of this as lots of different test cases being executed at once:

```
(base) wjones@CCU022633 source % python stencil_2d_tester.py
usage: stencil_2d_tester.py [-h] --testing-dir TESTING_DIR [--make MAKE] [--serial SERIAL]
                            [--pth PTH] --N1 N1 --N2 N2 --Nstep NSTEP --I1 I1 --I2 I2
                            --Istep ISTEP --T1 T1 --T2 T2 --Tstep TSTEP [--tol TOL]
                            [--endian {little,big}] [--keep]
stencil_2d_tester.py: error: the following arguments are required: --testing-dir, --N1, --N2, --Nstep,
--I1, --I2, --Istep, --T1, --T2, --Tstep
(base) wjones@CCU022633 source %
```

Here is an example of me testing mine out:

```
(base) wjones@CCU022633 source % python stencil_2d_tester.py \
--testing-dir ./testing_output \
--make ./make-2d \
--serial ./stencil-2d \
--pth ./stencil-2d-pth \
--N1 128 \
--N2 512 \
--Nstep 128 \
--I1 10 \
--I2 50 \
--Istep 20 \
--T1 1 \
--T2 12 \
--Tstep 1 \
--tol 0.0 \
--keep
[N= 128 I=  10 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
```

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
[N= 128 I=  10 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  10 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  30 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 128 I=  50 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  10 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  30 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 256 I=  50 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
```

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
[N= 384 I=  10 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  10 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  30 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 384 I=  50 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  10 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  30 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  1] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  2] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  3] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  4] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  5] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  6] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  7] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  8] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T=  9] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T= 10] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T= 11] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)
[N= 512 I=  50 T= 12] FINAL=OK (max|Δ|=0.000e+00); STACK=OK (max|Δ|=0.000e+00)

================ SUMMARY ================
Total cases:         144
Final matches:       144 / 144  (100.0%)
Stack matches:       144 / 144  (100.0%)
Artifacts/logs dir:  ./testing_output
CSV:                 ./testing_output/results.csv
```

```
Log:                ./testing_output/results.log
Elapsed:            8.31s
(base) wjones@CCU022633 source %
```

So we tested out a lot of different combinations – and it worked – so we're feeling good about this.

Now let's get some timing, speedup, efficiency and iso-efficiency results:

```
(base) wjones@CCU022633 source % python bench_stencil2d_pthreads.py --help
usage: bench_stencil2d_pthreads.py [-h] [--make_exe MAKE_EXE] [--pth_exe PTH_EXE]
                                   [--N_start N_START] [--N_max N_MAX] [--N1 N1] [--N2 N2]
                                   [--num_Ns NUM_NS] [--P_start P_START] [--P_step P_STEP]
                                   [--P_max P_MAX] [--Ps PS] [--I1 I1] [--I2 I2]
                                   [--Istep ISTEP] [--Is IS] [--warmup WARMUP]
                                   [--trials TRIALS] [--timeout_sec TIMEOUT_SEC]
                                   [--eff_range EFF_RANGE] [--eff_targets EFF_TARGETS]
                                   [--results-dir RESULTS_DIR] [--label LABEL]

Benchmark pthreads stencil-2d over N, P, and I (no stacks).

options:
  -h, --help            show this help message and exit
  --make_exe MAKE_EXE   path to make-2d (default: ./make-2d)
  --pth_exe PTH_EXE     path to stencil-2d-pth (default: ./stencil-2d-pth)
  --N_start N_START     doubling start N (if >0, use doubling) (default: 0)
  --N_max N_MAX         doubling max N (inclusive) (default: 0)
  --N1 N1               min N (evenly spaced) (default: 256)
  --N2 N2               max N (evenly spaced) (default: 4096)
  --num_Ns NUM_NS       number of N points (evenly spaced) (default: 5)
  --P_start P_START     P range start (default: 1)
  --P_step P_STEP       P range step (default: 1)
  --P_max P_MAX         P range max (inclusive) (default: 8)
  --Ps PS               explicit P list if you don't want range (default: [])
  --I1 I1               iterations min (default: 10)
  --I2 I2               iterations max (default: 50)
  --Istep ISTEP         iterations step (default: 20)
  --Is IS               explicit list of iterations (default: [])
  --warmup WARMUP       warmup runs per (N,P,I) not timed (default: 1)
  --trials TRIALS       timed trials per (N,P,I) (default: 5)
  --timeout_sec TIMEOUT_SEC
                        per-run timeout seconds (default: 600)
  --eff_range EFF_RANGE
                        e.g., "0.3 0.1" → 0.3,0.4,... (default: )
  --eff_targets EFF_TARGETS
                        e.g., 0.3,0.5,0.7,0.9 (default: )
  --results-dir RESULTS_DIR
                        folder for outputs (default: perf_results)
  --label LABEL         optional label suffix for filenames (default: )
(base) wjones@CCU022633 source %
```

Now let's run with some specific commands:

```
(base) wjones@CCU022633 source % python bench_stencil2d_pthreads.py \
--make_exe ./make-2d \
--pth_exe ./stencil-2d-pth \
--N1 128 \
--N2 256 \
--num_Ns 3 \
--P_start 1 \
```

```
--P_step 1 \
--P_max 12 \
--I1 10 \
--I2 200 \
--Istep 20 \
--warmup 1 \
--trials 4 \
--timeout_sec 600 \
--eff_range "0.29 0.05" \
--results-dir ./perf_results \
--label bench_run1
[    1/  360] N=   128 I=   10 P=  1 — trials=4
[    2/  360] N=   128 I=   10 P=  2 — trials=4
[    3/  360] N=   128 I=   10 P=  3 — trials=4
[    4/  360] N=   128 I=   10 P=  4 — trials=4
[    5/  360] N=   128 I=   10 P=  5 — trials=4
[    6/  360] N=   128 I=   10 P=  6 — trials=4
[    7/  360] N=   128 I=   10 P=  7 — trials=4
[    8/  360] N=   128 I=   10 P=  8 — trials=4
[    9/  360] N=   128 I=   10 P=  9 — trials=4
[   10/  360] N=   128 I=   10 P= 10 — trials=4
[   11/  360] N=   128 I=   10 P= 11 — trials=4
[   12/  360] N=   128 I=   10 P= 12 — trials=4
[   13/  360] N=   128 I=   30 P=  1 — trials=4
[   14/  360] N=   128 I=   30 P=  2 — trials=4
[   15/  360] N=   128 I=   30 P=  3 — trials=4
[   16/  360] N=   128 I=   30 P=  4 — trials=4
[   17/  360] N=   128 I=   30 P=  5 — trials=4
[   18/  360] N=   128 I=   30 P=  6 — trials=4
[   19/  360] N=   128 I=   30 P=  7 — trials=4
[   20/  360] N=   128 I=   30 P=  8 — trials=4
[   21/  360] N=   128 I=   30 P=  9 — trials=4
[   22/  360] N=   128 I=   30 P= 10 — trials=4
[   23/  360] N=   128 I=   30 P= 11 — trials=4
[   24/  360] N=   128 I=   30 P= 12 — trials=4
[   25/  360] N=   128 I=   50 P=  1 — trials=4
[   26/  360] N=   128 I=   50 P=  2 — trials=4
[   27/  360] N=   128 I=   50 P=  3 — trials=4
[   28/  360] N=   128 I=   50 P=  4 — trials=4
[   29/  360] N=   128 I=   50 P=  5 — trials=4
[   30/  360] N=   128 I=   50 P=  6 — trials=4
[   31/  360] N=   128 I=   50 P=  7 — trials=4
[   32/  360] N=   128 I=   50 P=  8 — trials=4
[   33/  360] N=   128 I=   50 P=  9 — trials=4
[   34/  360] N=   128 I=   50 P= 10 — trials=4
[   35/  360] N=   128 I=   50 P= 11 — trials=4
[   36/  360] N=   128 I=   50 P= 12 — trials=4
[   37/  360] N=   128 I=   70 P=  1 — trials=4
[   38/  360] N=   128 I=   70 P=  2 — trials=4
[   39/  360] N=   128 I=   70 P=  3 — trials=4
[   40/  360] N=   128 I=   70 P=  4 — trials=4
[   41/  360] N=   128 I=   70 P=  5 — trials=4
[   42/  360] N=   128 I=   70 P=  6 — trials=4
[   43/  360] N=   128 I=   70 P=  7 — trials=4
[   44/  360] N=   128 I=   70 P=  8 — trials=4
[   45/  360] N=   128 I=   70 P=  9 — trials=4
[   46/  360] N=   128 I=   70 P= 10 — trials=4
[   47/  360] N=   128 I=   70 P= 11 — trials=4
[   48/  360] N=   128 I=   70 P= 12 — trials=4
[   49/  360] N=   128 I=   90 P=  1 — trials=4
[   50/  360] N=   128 I=   90 P=  2 — trials=4
[   51/  360] N=   128 I=   90 P=  3 — trials=4
[   52/  360] N=   128 I=   90 P=  4 — trials=4
[   53/  360] N=   128 I=   90 P=  5 — trials=4
[   54/  360] N=   128 I=   90 P=  6 — trials=4
[   55/  360] N=   128 I=   90 P=  7 — trials=4
[   56/  360] N=   128 I=   90 P=  8 — trials=4
[   57/  360] N=   128 I=   90 P=  9 — trials=4
[   58/  360] N=   128 I=   90 P= 10 — trials=4
[   59/  360] N=   128 I=   90 P= 11 — trials=4
[   60/  360] N=   128 I=   90 P= 12 — trials=4
```

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
[   61/  360] N=   128 I=  110 P=  1 — trials=4
[   62/  360] N=   128 I=  110 P=  2 — trials=4
[   63/  360] N=   128 I=  110 P=  3 — trials=4
[   64/  360] N=   128 I=  110 P=  4 — trials=4
[   65/  360] N=   128 I=  110 P=  5 — trials=4
[   66/  360] N=   128 I=  110 P=  6 — trials=4
[   67/  360] N=   128 I=  110 P=  7 — trials=4
[   68/  360] N=   128 I=  110 P=  8 — trials=4
[   69/  360] N=   128 I=  110 P=  9 — trials=4
[   70/  360] N=   128 I=  110 P= 10 — trials=4
[   71/  360] N=   128 I=  110 P= 11 — trials=4
[   72/  360] N=   128 I=  110 P= 12 — trials=4
[   73/  360] N=   128 I=  130 P=  1 — trials=4
[   74/  360] N=   128 I=  130 P=  2 — trials=4
[   75/  360] N=   128 I=  130 P=  3 — trials=4
[   76/  360] N=   128 I=  130 P=  4 — trials=4
[   77/  360] N=   128 I=  130 P=  5 — trials=4
[   78/  360] N=   128 I=  130 P=  6 — trials=4
[   79/  360] N=   128 I=  130 P=  7 — trials=4
[   80/  360] N=   128 I=  130 P=  8 — trials=4
[   81/  360] N=   128 I=  130 P=  9 — trials=4
[   82/  360] N=   128 I=  130 P= 10 — trials=4
[   83/  360] N=   128 I=  130 P= 11 — trials=4
[   84/  360] N=   128 I=  130 P= 12 — trials=4
[   85/  360] N=   128 I=  150 P=  1 — trials=4
[   86/  360] N=   128 I=  150 P=  2 — trials=4
[   87/  360] N=   128 I=  150 P=  3 — trials=4
[   88/  360] N=   128 I=  150 P=  4 — trials=4
[   89/  360] N=   128 I=  150 P=  5 — trials=4
[   90/  360] N=   128 I=  150 P=  6 — trials=4
[   91/  360] N=   128 I=  150 P=  7 — trials=4
[   92/  360] N=   128 I=  150 P=  8 — trials=4
[   93/  360] N=   128 I=  150 P=  9 — trials=4
[   94/  360] N=   128 I=  150 P= 10 — trials=4
[   95/  360] N=   128 I=  150 P= 11 — trials=4
[   96/  360] N=   128 I=  150 P= 12 — trials=4
[   97/  360] N=   128 I=  170 P=  1 — trials=4
[   98/  360] N=   128 I=  170 P=  2 — trials=4
[   99/  360] N=   128 I=  170 P=  3 — trials=4
[  100/  360] N=   128 I=  170 P=  4 — trials=4
[  101/  360] N=   128 I=  170 P=  5 — trials=4
[  102/  360] N=   128 I=  170 P=  6 — trials=4
[  103/  360] N=   128 I=  170 P=  7 — trials=4
[  104/  360] N=   128 I=  170 P=  8 — trials=4
[  105/  360] N=   128 I=  170 P=  9 — trials=4
[  106/  360] N=   128 I=  170 P= 10 — trials=4
[  107/  360] N=   128 I=  170 P= 11 — trials=4
[  108/  360] N=   128 I=  170 P= 12 — trials=4
[  109/  360] N=   128 I=  190 P=  1 — trials=4
[  110/  360] N=   128 I=  190 P=  2 — trials=4
[  111/  360] N=   128 I=  190 P=  3 — trials=4
[  112/  360] N=   128 I=  190 P=  4 — trials=4
[  113/  360] N=   128 I=  190 P=  5 — trials=4
[  114/  360] N=   128 I=  190 P=  6 — trials=4
[  115/  360] N=   128 I=  190 P=  7 — trials=4
[  116/  360] N=   128 I=  190 P=  8 — trials=4
[  117/  360] N=   128 I=  190 P=  9 — trials=4
[  118/  360] N=   128 I=  190 P= 10 — trials=4
[  119/  360] N=   128 I=  190 P= 11 — trials=4
[  120/  360] N=   128 I=  190 P= 12 — trials=4
[  121/  360] N=   192 I=   10 P=  1 — trials=4
[  122/  360] N=   192 I=   10 P=  2 — trials=4
[  123/  360] N=   192 I=   10 P=  3 — trials=4
[  124/  360] N=   192 I=   10 P=  4 — trials=4
[  125/  360] N=   192 I=   10 P=  5 — trials=4
[  126/  360] N=   192 I=   10 P=  6 — trials=4
[  127/  360] N=   192 I=   10 P=  7 — trials=4
[  128/  360] N=   192 I=   10 P=  8 — trials=4
[  129/  360] N=   192 I=   10 P=  9 — trials=4
[  130/  360] N=   192 I=   10 P= 10 — trials=4
[  131/  360] N=   192 I=   10 P= 11 — trials=4
```

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
[  132/  360] N=   192 I=   10 P= 12 — trials=4
[  133/  360] N=   192 I=   30 P=  1 — trials=4
[  134/  360] N=   192 I=   30 P=  2 — trials=4
[  135/  360] N=   192 I=   30 P=  3 — trials=4
[  136/  360] N=   192 I=   30 P=  4 — trials=4
[  137/  360] N=   192 I=   30 P=  5 — trials=4
[  138/  360] N=   192 I=   30 P=  6 — trials=4
[  139/  360] N=   192 I=   30 P=  7 — trials=4
[  140/  360] N=   192 I=   30 P=  8 — trials=4
[  141/  360] N=   192 I=   30 P=  9 — trials=4
[  142/  360] N=   192 I=   30 P= 10 — trials=4
[  143/  360] N=   192 I=   30 P= 11 — trials=4
[  144/  360] N=   192 I=   30 P= 12 — trials=4
[  145/  360] N=   192 I=   50 P=  1 — trials=4
[  146/  360] N=   192 I=   50 P=  2 — trials=4
[  147/  360] N=   192 I=   50 P=  3 — trials=4
[  148/  360] N=   192 I=   50 P=  4 — trials=4
[  149/  360] N=   192 I=   50 P=  5 — trials=4
[  150/  360] N=   192 I=   50 P=  6 — trials=4
[  151/  360] N=   192 I=   50 P=  7 — trials=4
[  152/  360] N=   192 I=   50 P=  8 — trials=4
[  153/  360] N=   192 I=   50 P=  9 — trials=4
[  154/  360] N=   192 I=   50 P= 10 — trials=4
[  155/  360] N=   192 I=   50 P= 11 — trials=4
[  156/  360] N=   192 I=   50 P= 12 — trials=4
[  157/  360] N=   192 I=   70 P=  1 — trials=4
[  158/  360] N=   192 I=   70 P=  2 — trials=4
[  159/  360] N=   192 I=   70 P=  3 — trials=4
[  160/  360] N=   192 I=   70 P=  4 — trials=4
[  161/  360] N=   192 I=   70 P=  5 — trials=4
[  162/  360] N=   192 I=   70 P=  6 — trials=4
…. // REMOVED SOME OF THE OUTPUT TO THE SCREEN SO THIS DOESN'T TAKE UP SO MUCH SPACE
[  340/  360] N=   256 I=  170 P=  4 — trials=4
[  341/  360] N=   256 I=  170 P=  5 — trials=4
[  342/  360] N=   256 I=  170 P=  6 — trials=4
[  343/  360] N=   256 I=  170 P=  7 — trials=4
[  344/  360] N=   256 I=  170 P=  8 — trials=4
[  345/  360] N=   256 I=  170 P=  9 — trials=4
[  346/  360] N=   256 I=  170 P= 10 — trials=4
[  347/  360] N=   256 I=  170 P= 11 — trials=4
[  348/  360] N=   256 I=  170 P= 12 — trials=4
[  349/  360] N=   256 I=  190 P=  1 — trials=4
[  350/  360] N=   256 I=  190 P=  2 — trials=4
[  351/  360] N=   256 I=  190 P=  3 — trials=4
[  352/  360] N=   256 I=  190 P=  4 — trials=4
[  353/  360] N=   256 I=  190 P=  5 — trials=4
[  354/  360] N=   256 I=  190 P=  6 — trials=4
[  355/  360] N=   256 I=  190 P=  7 — trials=4
[  356/  360] N=   256 I=  190 P=  8 — trials=4
[  357/  360] N=   256 I=  190 P=  9 — trials=4
[  358/  360] N=   256 I=  190 P= 10 — trials=4
[  359/  360] N=   256 I=  190 P= 11 — trials=4
[  360/  360] N=   256 I=  190 P= 12 — trials=4

Done.
Raw CSV : perf_results/raw_runs-bench_run1.csv
Summary : perf_results/summary-bench_run1.csv
Report  : perf_results/report-bench_run1.txt
Plots   :
  perf_results/timing_vs_P_by_N_I10.png
  perf_results/speedup_vs_P_by_N_I10.png
  perf_results/efficiency_vs_P_by_N_I10.png
  perf_results/efficiency_heatmap_I10.png
  perf_results/isoefficiency_E=0.29_I10.png
  perf_results/isoefficiency_E=0.29_I10.csv
  perf_results/isoefficiency_E=0.34_I10.png
  perf_results/isoefficiency_E=0.34_I10.csv
  perf_results/isoefficiency_E=0.39_I10.png
  perf_results/isoefficiency_E=0.39_I10.csv
  perf_results/isoefficiency_E=0.44_I10.png
  perf_results/isoefficiency_E=0.44_I10.csv
```

```
perf_results/isoefficiency_E=0.49_I10.png
perf_results/isoefficiency_E=0.49_I10.csv
perf_results/isoefficiency_E=0.54_I10.png
perf_results/isoefficiency_E=0.54_I10.csv
perf_results/isoefficiency_E=0.59_I10.png
perf_results/isoefficiency_E=0.59_I10.csv
perf_results/isoefficiency_E=0.64_I10.png
perf_results/isoefficiency_E=0.64_I10.csv
perf_results/isoefficiency_E=0.69_I10.png
perf_results/isoefficiency_E=0.69_I10.csv
perf_results/isoefficiency_E=0.74_I10.png
perf_results/isoefficiency_E=0.74_I10.csv
perf_results/isoefficiency_E=0.79_I10.png
perf_results/isoefficiency_E=0.79_I10.csv
perf_results/isoefficiency_E=0.84_I10.png
perf_results/isoefficiency_E=0.84_I10.csv
perf_results/isoefficiency_E=0.89_I10.png
perf_results/isoefficiency_E=0.89_I10.csv
perf_results/isoefficiency_E=0.94_I10.png
perf_results/isoefficiency_E=0.94_I10.csv
perf_results/isoefficiency_E=0.99_I10.png
perf_results/isoefficiency_E=0.99_I10.csv
perf_results/isoefficiency_surface_I10.png
perf_results/timing_vs_P_by_N_I30.png
perf_results/speedup_vs_P_by_N_I30.png
perf_results/efficiency_vs_P_by_N_I30.png
perf_results/efficiency_heatmap_I30.png
perf_results/isoefficiency_E=0.29_I30.png
perf_results/isoefficiency_E=0.29_I30.csv
perf_results/isoefficiency_E=0.34_I30.png
perf_results/isoefficiency_E=0.34_I30.csv
perf_results/isoefficiency_E=0.39_I30.png
perf_results/isoefficiency_E=0.39_I30.csv
perf_results/isoefficiency_E=0.44_I30.png
perf_results/isoefficiency_E=0.44_I30.csv
perf_results/isoefficiency_E=0.49_I30.png
perf_results/isoefficiency_E=0.49_I30.csv
perf_results/isoefficiency_E=0.54_I30.png
perf_results/isoefficiency_E=0.54_I30.csv
perf_results/isoefficiency_E=0.59_I30.png
perf_results/isoefficiency_E=0.59_I30.csv
perf_results/isoefficiency_E=0.64_I30.png
perf_results/isoefficiency_E=0.64_I30.csv
perf_results/isoefficiency_E=0.69_I30.png
perf_results/isoefficiency_E=0.69_I30.csv
perf_results/isoefficiency_E=0.74_I30.png
perf_results/isoefficiency_E=0.74_I30.csv
perf_results/isoefficiency_E=0.79_I30.png
perf_results/isoefficiency_E=0.79_I30.csv
perf_results/isoefficiency_E=0.84_I30.png
perf_results/isoefficiency_E=0.84_I30.csv
perf_results/isoefficiency_E=0.89_I30.png
perf_results/isoefficiency_E=0.89_I30.csv
perf_results/isoefficiency_E=0.94_I30.png
perf_results/isoefficiency_E=0.94_I30.csv
perf_results/isoefficiency_E=0.99_I30.png
perf_results/isoefficiency_E=0.99_I30.csv
perf_results/isoefficiency_surface_I30.png
perf_results/timing_vs_P_by_N_I50.png
perf_results/speedup_vs_P_by_N_I50.png
perf_results/efficiency_vs_P_by_N_I50.png
perf_results/efficiency_heatmap_I50.png
perf_results/isoefficiency_E=0.29_I50.png
perf_results/isoefficiency_E=0.29_I50.csv
perf_results/isoefficiency_E=0.34_I50.png
perf_results/isoefficiency_E=0.34_I50.csv
perf_results/isoefficiency_E=0.39_I50.png
perf_results/isoefficiency_E=0.39_I50.csv
perf_results/isoefficiency_E=0.44_I50.png
perf_results/isoefficiency_E=0.44_I50.csv
perf_results/isoefficiency_E=0.49_I50.png
```

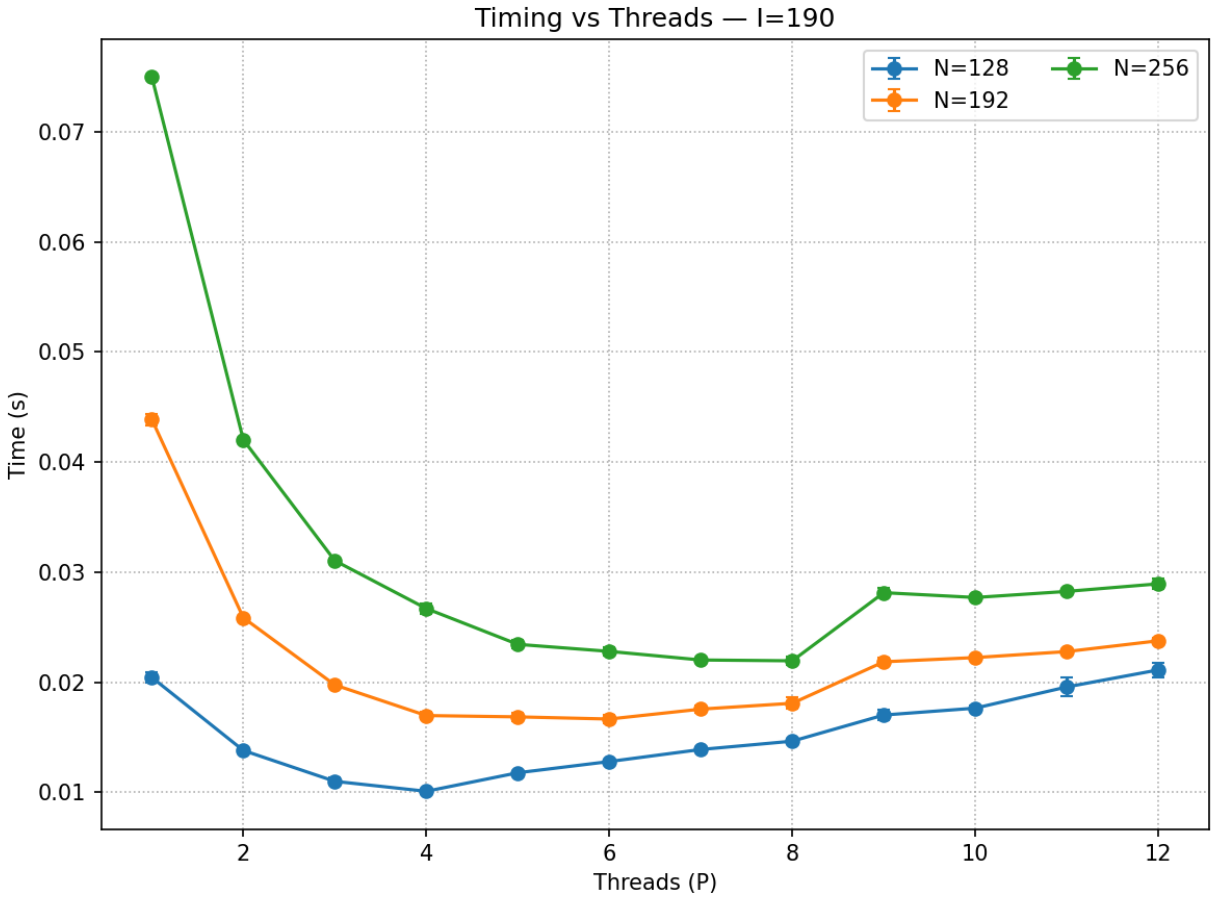
```
perf_results/isoefficiency_E=0.49_I50.csv
perf_results/isoefficiency_E=0.54_I50.png
perf_results/isoefficiency_E=0.54_I50.csv
perf_results/isoefficiency_E=0.59_I50.png
perf_results/isoefficiency_E=0.59_I50.csv
perf_results/isoefficiency_E=0.64_I50.png
perf_results/isoefficiency_E=0.64_I50.csv
perf_results/isoefficiency_E=0.69_I50.png
perf_results/isoefficiency_E=0.69_I50.csv
perf_results/isoefficiency_E=0.74_I50.png
perf_results/isoefficiency_E=0.74_I50.csv
perf_results/isoefficiency_E=0.79_I50.png
perf_results/isoefficiency_E=0.79_I50.csv
perf_results/isoefficiency_E=0.84_I50.png
perf_results/isoefficiency_E=0.84_I50.csv
perf_results/isoefficiency_E=0.89_I50.png
perf_results/isoefficiency_E=0.89_I50.csv
perf_results/isoefficiency_E=0.94_I50.png
perf_results/isoefficiency_E=0.94_I50.csv
perf_results/isoefficiency_E=0.99_I50.png
perf_results/isoefficiency_E=0.99_I50.csv
perf_results/isoefficiency_surface_I50.png
perf_results/timing_vs_P_by_N_I70.png
perf_results/speedup_vs_P_by_N_I70.png
perf_results/efficiency_vs_P_by_N_I70.png
perf_results/efficiency_heatmap_I70.png
perf_results/isoefficiency_E=0.29_I70.png
perf_results/isoefficiency_E=0.29_I70.csv
perf_results/isoefficiency_E=0.34_I70.png
perf_results/isoefficiency_E=0.34_I70.csv
perf_results/isoefficiency_E=0.39_I70.png
perf_results/isoefficiency_E=0.39_I70.csv
perf_results/isoefficiency_E=0.44_I70.png
perf_results/isoefficiency_E=0.44_I70.csv
perf_results/isoefficiency_E=0.49_I70.png
perf_results/isoefficiency_E=0.49_I70.csv
perf_results/isoefficiency_E=0.54_I70.png
perf_results/isoefficiency_E=0.54_I70.csv
perf_results/isoefficiency_E=0.59_I70.png
perf_results/isoefficiency_E=0.59_I70.csv
perf_results/isoefficiency_E=0.64_I70.png
perf_results/isoefficiency_E=0.64_I70.csv
perf_results/isoefficiency_E=0.69_I70.png
perf_results/isoefficiency_E=0.69_I70.csv
perf_results/isoefficiency_E=0.74_I70.png
perf_results/isoefficiency_E=0.74_I70.csv
perf_results/isoefficiency_E=0.79_I70.png
perf_results/isoefficiency_E=0.79_I70.csv
perf_results/isoefficiency_E=0.84_I70.png
perf_results/isoefficiency_E=0.84_I70.csv
perf_results/isoefficiency_E=0.89_I70.png
perf_results/isoefficiency_E=0.89_I70.csv
perf_results/isoefficiency_E=0.94_I70.png
perf_results/isoefficiency_E=0.94_I70.csv
perf_results/isoefficiency_E=0.99_I70.png
perf_results/isoefficiency_E=0.99_I70.csv
perf_results/isoefficiency_surface_I70.png
perf_results/timing_vs_P_by_N_I90.png
perf_results/speedup_vs_P_by_N_I90.png
perf_results/efficiency_vs_P_by_N_I90.png
perf_results/efficiency_heatmap_I90.png
perf_results/isoefficiency_E=0.29_I90.png
perf_results/isoefficiency_E=0.29_I90.csv
perf_results/isoefficiency_E=0.34_I90.png
perf_results/isoefficiency_E=0.34_I90.csv
perf_results/isoefficiency_E=0.39_I90.png
perf_results/isoefficiency_E=0.39_I90.csv
perf_results/isoefficiency_E=0.44_I90.png
perf_results/isoefficiency_E=0.44_I90.csv
perf_results/isoefficiency_E=0.49_I90.png
perf_results/isoefficiency_E=0.49_I90.csv
```

```
perf_results/isoefficiency_E=0.54_I90.png
perf_results/isoefficiency_E=0.54_I90.csv
perf_results/isoefficiency_E=0.59_I90.png
perf_results/isoefficiency_E=0.59_I90.csv
perf_results/isoefficiency_E=0.64_I90.png
perf_results/isoefficiency_E=0.64_I90.csv
perf_results/isoefficiency_E=0.69_I90.png
perf_results/isoefficiency_E=0.69_I90.csv
perf_results/isoefficiency_E=0.74_I90.png
perf_results/isoefficiency_E=0.74_I90.csv
perf_results/isoefficiency_E=0.79_I90.png
perf_results/isoefficiency_E=0.79_I90.csv
perf_results/isoefficiency_E=0.84_I90.png
perf_results/isoefficiency_E=0.84_I90.csv
perf_results/isoefficiency_E=0.89_I90.png
perf_results/isoefficiency_E=0.89_I90.csv
perf_results/isoefficiency_E=0.94_I90.png
perf_results/isoefficiency_E=0.94_I90.csv
perf_results/isoefficiency_E=0.99_I90.png
perf_results/isoefficiency_E=0.99_I90.csv
perf_results/isoefficiency_surface_I90.png
perf_results/timing_vs_P_by_N_I110.png
perf_results/speedup_vs_P_by_N_I110.png
perf_results/efficiency_vs_P_by_N_I110.png
perf_results/efficiency_heatmap_I110.png
perf_results/isoefficiency_E=0.29_I110.png
perf_results/isoefficiency_E=0.29_I110.csv
perf_results/isoefficiency_E=0.34_I110.png
perf_results/isoefficiency_E=0.34_I110.csv
perf_results/isoefficiency_E=0.39_I110.png
perf_results/isoefficiency_E=0.39_I110.csv
perf_results/isoefficiency_E=0.44_I110.png
perf_results/isoefficiency_E=0.44_I110.csv
perf_results/isoefficiency_E=0.49_I110.png
perf_results/isoefficiency_E=0.49_I110.csv
perf_results/isoefficiency_E=0.54_I110.png
perf_results/isoefficiency_E=0.54_I110.csv
perf_results/isoefficiency_E=0.59_I110.png
perf_results/isoefficiency_E=0.59_I110.csv
perf_results/isoefficiency_E=0.64_I110.png
perf_results/isoefficiency_E=0.64_I110.csv
perf_results/isoefficiency_E=0.69_I110.png
perf_results/isoefficiency_E=0.69_I110.csv
perf_results/isoefficiency_E=0.74_I110.png
perf_results/isoefficiency_E=0.74_I110.csv
perf_results/isoefficiency_E=0.79_I110.png
perf_results/isoefficiency_E=0.79_I110.csv
perf_results/isoefficiency_E=0.84_I110.png
perf_results/isoefficiency_E=0.84_I110.csv
perf_results/isoefficiency_E=0.89_I110.png
perf_results/isoefficiency_E=0.89_I110.csv
perf_results/isoefficiency_E=0.94_I110.png
perf_results/isoefficiency_E=0.94_I110.csv
perf_results/isoefficiency_E=0.99_I110.png
perf_results/isoefficiency_E=0.99_I110.csv
perf_results/isoefficiency_surface_I110.png
perf_results/timing_vs_P_by_N_I130.png
perf_results/speedup_vs_P_by_N_I130.png
perf_results/efficiency_vs_P_by_N_I130.png
perf_results/efficiency_heatmap_I130.png
perf_results/isoefficiency_E=0.29_I130.png
perf_results/isoefficiency_E=0.29_I130.csv
perf_results/isoefficiency_E=0.34_I130.png
perf_results/isoefficiency_E=0.34_I130.csv
perf_results/isoefficiency_E=0.39_I130.png
perf_results/isoefficiency_E=0.39_I130.csv
perf_results/isoefficiency_E=0.44_I130.png
perf_results/isoefficiency_E=0.44_I130.csv
perf_results/isoefficiency_E=0.49_I130.png
perf_results/isoefficiency_E=0.49_I130.csv
perf_results/isoefficiency_E=0.54_I130.png
```

```
perf_results/isoefficiency_E=0.54_I130.csv
perf_results/isoefficiency_E=0.59_I130.png
perf_results/isoefficiency_E=0.59_I130.csv
perf_results/isoefficiency_E=0.64_I130.png
perf_results/isoefficiency_E=0.64_I130.csv
perf_results/isoefficiency_E=0.69_I130.png
perf_results/isoefficiency_E=0.69_I130.csv
perf_results/isoefficiency_E=0.74_I130.png
perf_results/isoefficiency_E=0.74_I130.csv
perf_results/isoefficiency_E=0.79_I130.png
perf_results/isoefficiency_E=0.79_I130.csv
perf_results/isoefficiency_E=0.84_I130.png
perf_results/isoefficiency_E=0.84_I130.csv
perf_results/isoefficiency_E=0.89_I130.png
perf_results/isoefficiency_E=0.89_I130.csv
perf_results/isoefficiency_E=0.94_I130.png
perf_results/isoefficiency_E=0.94_I130.csv
perf_results/isoefficiency_E=0.99_I130.png
perf_results/isoefficiency_E=0.99_I130.csv
perf_results/isoefficiency_surface_I130.png
perf_results/timing_vs_P_by_N_I150.png
perf_results/speedup_vs_P_by_N_I150.png
perf_results/efficiency_vs_P_by_N_I150.png
perf_results/efficiency_heatmap_I150.png
perf_results/isoefficiency_E=0.29_I150.png
perf_results/isoefficiency_E=0.29_I150.csv
perf_results/isoefficiency_E=0.34_I150.png
perf_results/isoefficiency_E=0.34_I150.csv
perf_results/isoefficiency_E=0.39_I150.png
perf_results/isoefficiency_E=0.39_I150.csv
perf_results/isoefficiency_E=0.44_I150.png
perf_results/isoefficiency_E=0.44_I150.csv
perf_results/isoefficiency_E=0.49_I150.png
perf_results/isoefficiency_E=0.49_I150.csv
perf_results/isoefficiency_E=0.54_I150.png
perf_results/isoefficiency_E=0.54_I150.csv
perf_results/isoefficiency_E=0.59_I150.png
perf_results/isoefficiency_E=0.59_I150.csv
perf_results/isoefficiency_E=0.64_I150.png
perf_results/isoefficiency_E=0.64_I150.csv
perf_results/isoefficiency_E=0.69_I150.png
perf_results/isoefficiency_E=0.69_I150.csv
perf_results/isoefficiency_E=0.74_I150.png
perf_results/isoefficiency_E=0.74_I150.csv
perf_results/isoefficiency_E=0.79_I150.png
perf_results/isoefficiency_E=0.79_I150.csv
perf_results/isoefficiency_E=0.84_I150.png
perf_results/isoefficiency_E=0.84_I150.csv
perf_results/isoefficiency_E=0.89_I150.png
perf_results/isoefficiency_E=0.89_I150.csv
perf_results/isoefficiency_E=0.94_I150.png
perf_results/isoefficiency_E=0.94_I150.csv
perf_results/isoefficiency_E=0.99_I150.png
perf_results/isoefficiency_E=0.99_I150.csv
perf_results/isoefficiency_surface_I150.png
perf_results/timing_vs_P_by_N_I170.png
perf_results/speedup_vs_P_by_N_I170.png
perf_results/efficiency_vs_P_by_N_I170.png
perf_results/efficiency_heatmap_I170.png
perf_results/isoefficiency_E=0.29_I170.png
perf_results/isoefficiency_E=0.29_I170.csv
perf_results/isoefficiency_E=0.34_I170.png
perf_results/isoefficiency_E=0.34_I170.csv
perf_results/isoefficiency_E=0.39_I170.png
perf_results/isoefficiency_E=0.39_I170.csv
perf_results/isoefficiency_E=0.44_I170.png
perf_results/isoefficiency_E=0.44_I170.csv
perf_results/isoefficiency_E=0.49_I170.png
perf_results/isoefficiency_E=0.49_I170.csv
perf_results/isoefficiency_E=0.54_I170.png
perf_results/isoefficiency_E=0.54_I170.csv
```

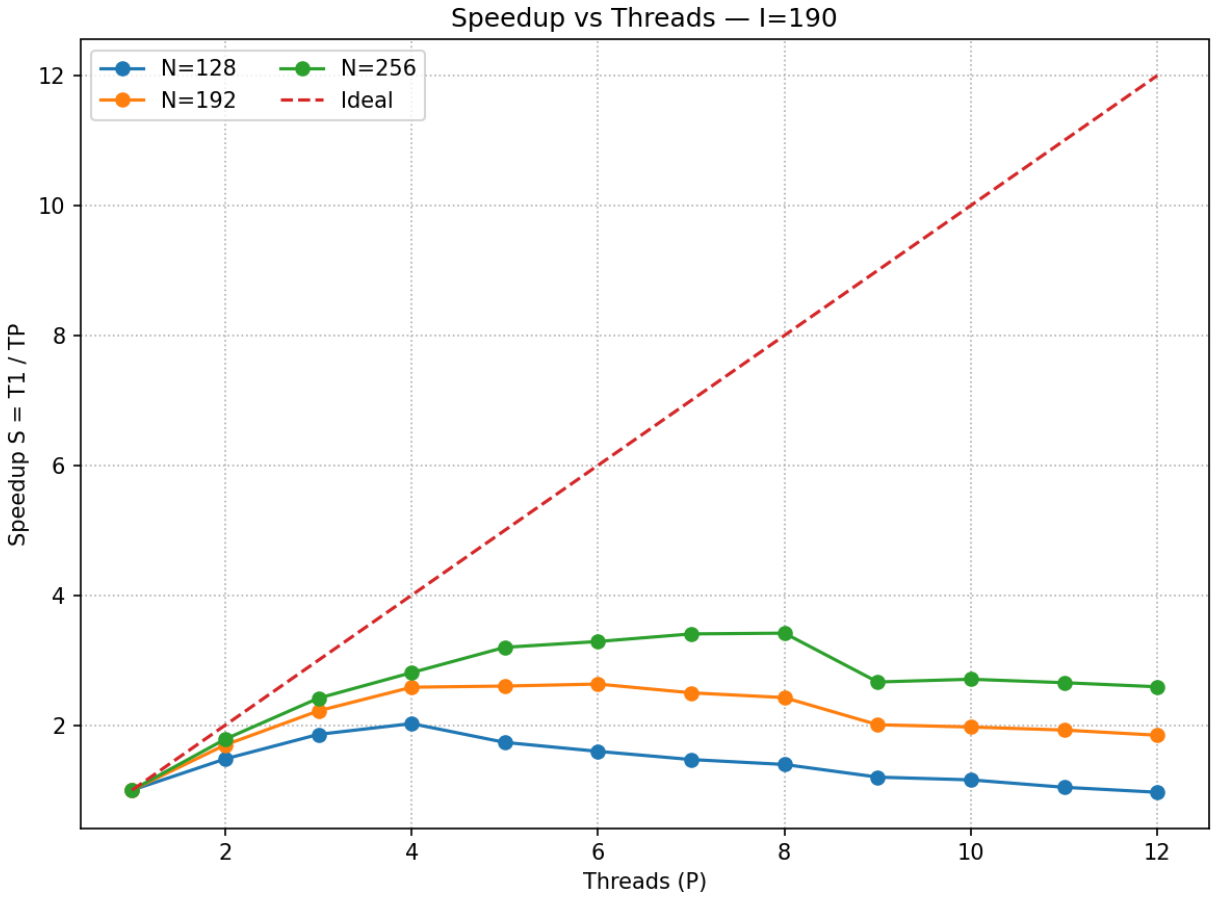
```
  perf_results/isoefficiency_E=0.59_I170.png
  perf_results/isoefficiency_E=0.59_I170.csv
  perf_results/isoefficiency_E=0.64_I170.png
  perf_results/isoefficiency_E=0.64_I170.csv
  perf_results/isoefficiency_E=0.69_I170.png
  perf_results/isoefficiency_E=0.69_I170.csv
  perf_results/isoefficiency_E=0.74_I170.png
  perf_results/isoefficiency_E=0.74_I170.csv
  perf_results/isoefficiency_E=0.79_I170.png
  perf_results/isoefficiency_E=0.79_I170.csv
  perf_results/isoefficiency_E=0.84_I170.png
  perf_results/isoefficiency_E=0.84_I170.csv
  perf_results/isoefficiency_E=0.89_I170.png
  perf_results/isoefficiency_E=0.89_I170.csv
  perf_results/isoefficiency_E=0.94_I170.png
  perf_results/isoefficiency_E=0.94_I170.csv
  perf_results/isoefficiency_E=0.99_I170.png
  perf_results/isoefficiency_E=0.99_I170.csv
  perf_results/isoefficiency_surface_I170.png
  perf_results/timing_vs_P_by_N_I190.png
  perf_results/speedup_vs_P_by_N_I190.png
  perf_results/efficiency_vs_P_by_N_I190.png
  perf_results/efficiency_heatmap_I190.png
  perf_results/isoefficiency_E=0.29_I190.png
  perf_results/isoefficiency_E=0.29_I190.csv
  perf_results/isoefficiency_E=0.34_I190.png
  perf_results/isoefficiency_E=0.34_I190.csv
  perf_results/isoefficiency_E=0.39_I190.png
  perf_results/isoefficiency_E=0.39_I190.csv
  perf_results/isoefficiency_E=0.44_I190.png
  perf_results/isoefficiency_E=0.44_I190.csv
  perf_results/isoefficiency_E=0.49_I190.png
  perf_results/isoefficiency_E=0.49_I190.csv
  perf_results/isoefficiency_E=0.54_I190.png
  perf_results/isoefficiency_E=0.54_I190.csv
  perf_results/isoefficiency_E=0.59_I190.png
  perf_results/isoefficiency_E=0.59_I190.csv
  perf_results/isoefficiency_E=0.64_I190.png
  perf_results/isoefficiency_E=0.64_I190.csv
  perf_results/isoefficiency_E=0.69_I190.png
  perf_results/isoefficiency_E=0.69_I190.csv
  perf_results/isoefficiency_E=0.74_I190.png
  perf_results/isoefficiency_E=0.74_I190.csv
  perf_results/isoefficiency_E=0.79_I190.png
  perf_results/isoefficiency_E=0.79_I190.csv
  perf_results/isoefficiency_E=0.84_I190.png
  perf_results/isoefficiency_E=0.84_I190.csv
  perf_results/isoefficiency_E=0.89_I190.png
  perf_results/isoefficiency_E=0.89_I190.csv
  perf_results/isoefficiency_E=0.94_I190.png
  perf_results/isoefficiency_E=0.94_I190.csv
  perf_results/isoefficiency_E=0.99_I190.png
  perf_results/isoefficiency_E=0.99_I190.csv
  perf_results/isoefficiency_surface_I190.png
(base) wjones@CCU022633 source %
```

Specifically – we see (in this short run on my laptop):

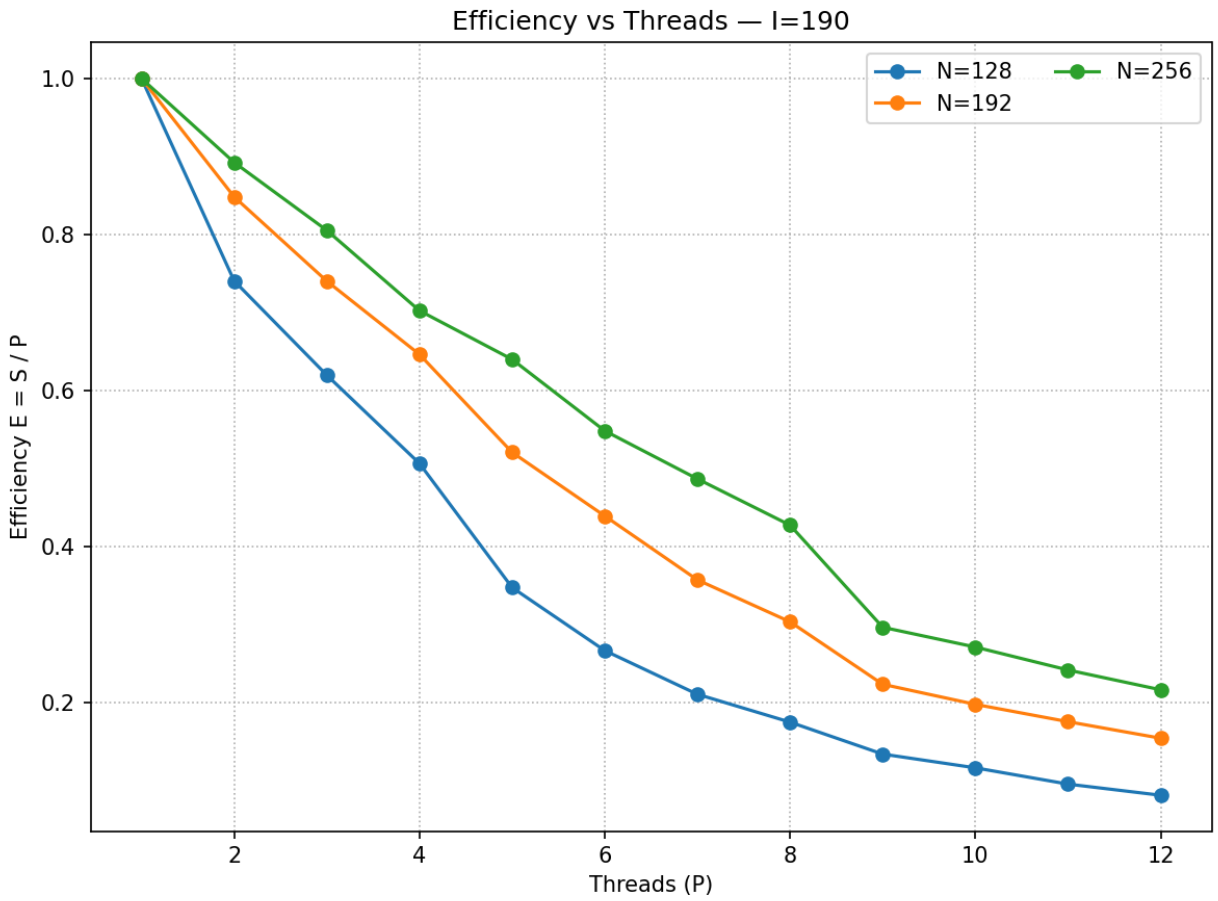


(the size of the file are so small – and the number of iterations – it makes it really bad in terms of performance)

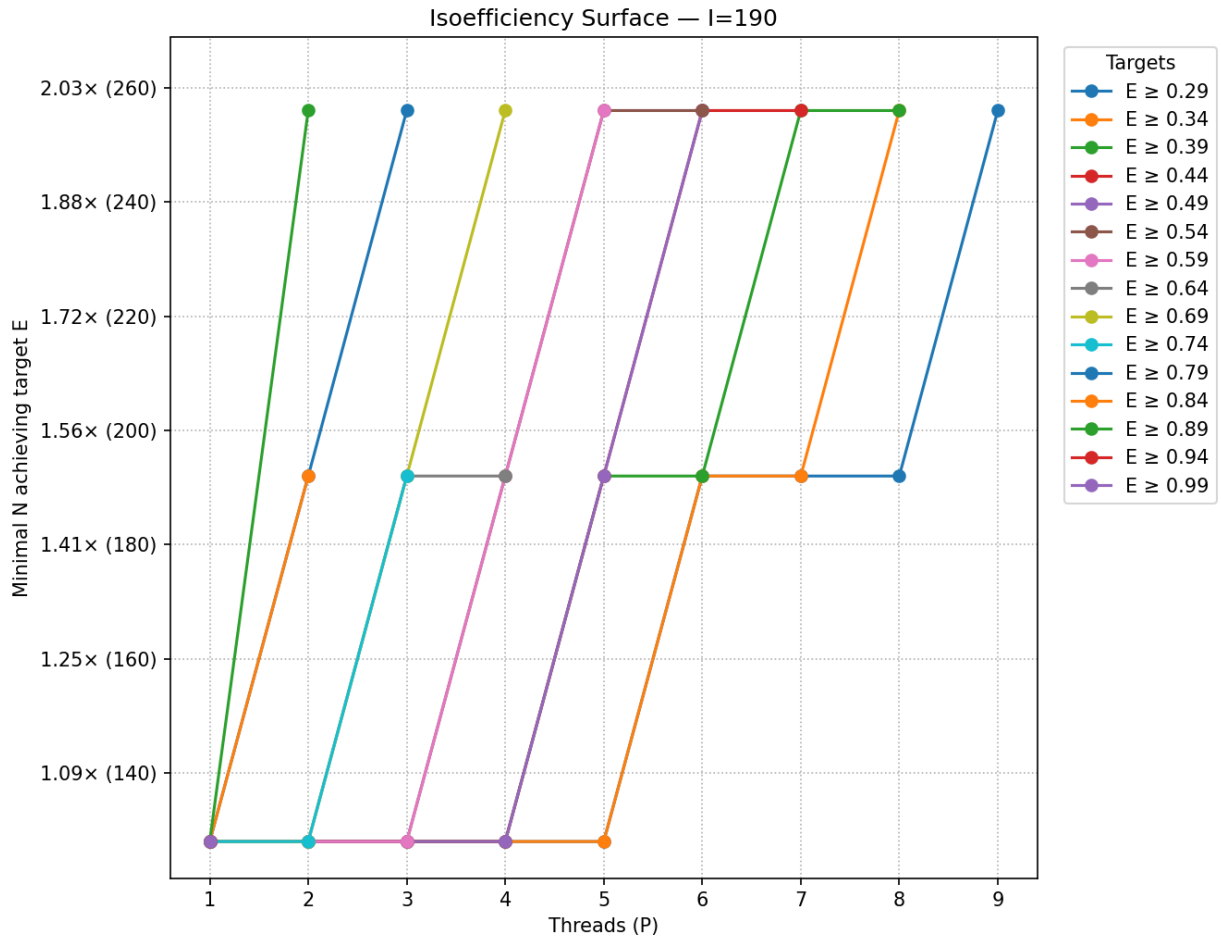
Parallel (using P-threads) 9-Point 2D Stencil Kernel



Parallel (using P-threads) 9-Point 2D Stencil Kernel

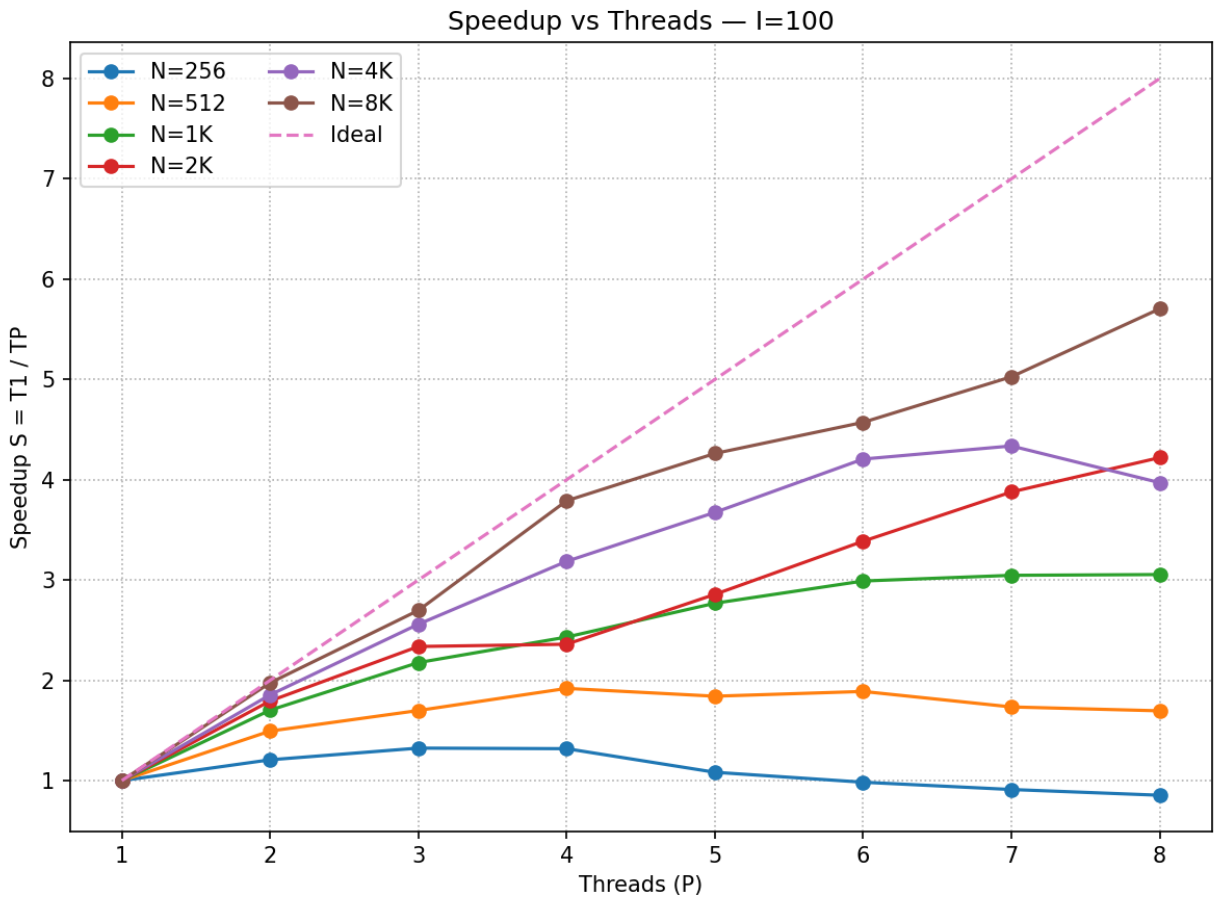


Parallel (using P-threads) 9-Point 2D Stencil Kernel

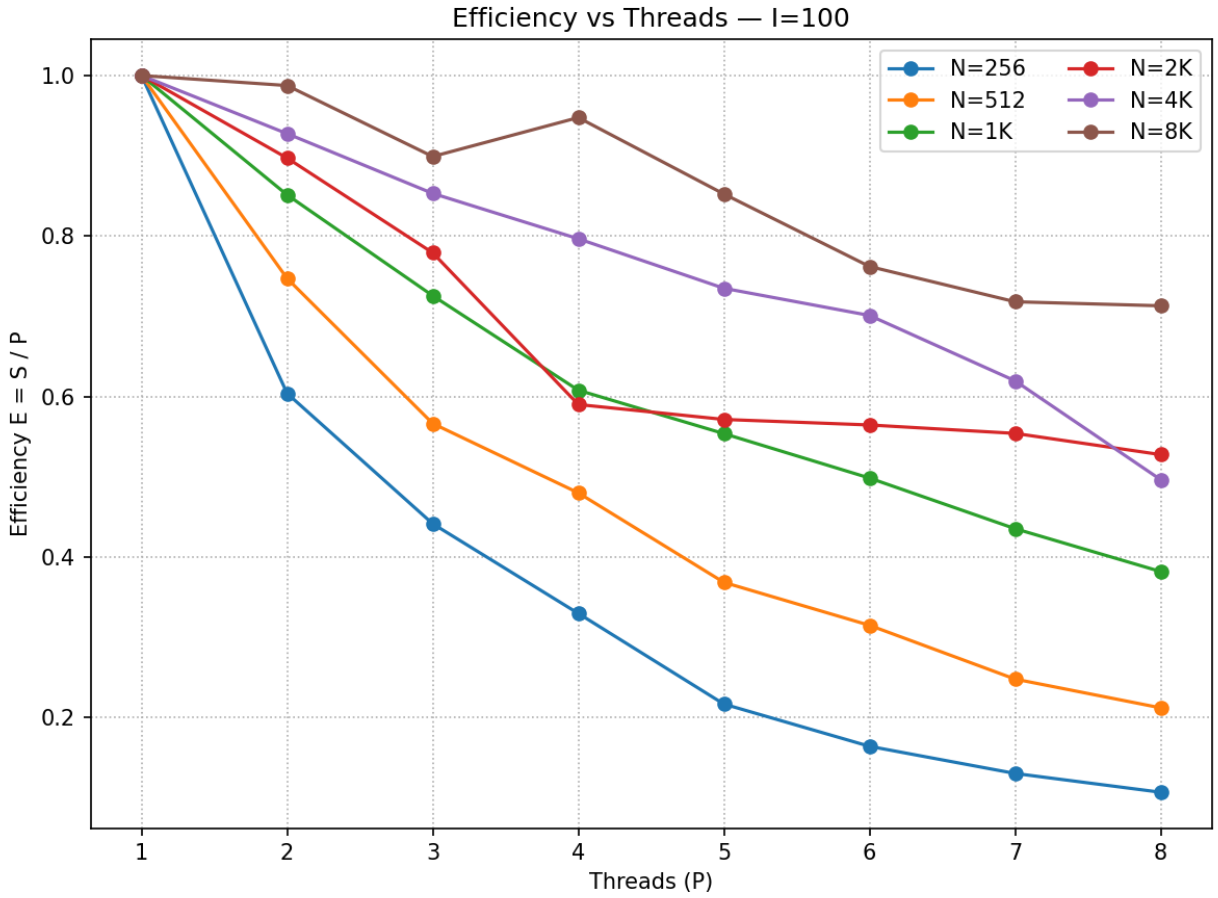


Now – if we were to run higher resolution plates, we get some better results:

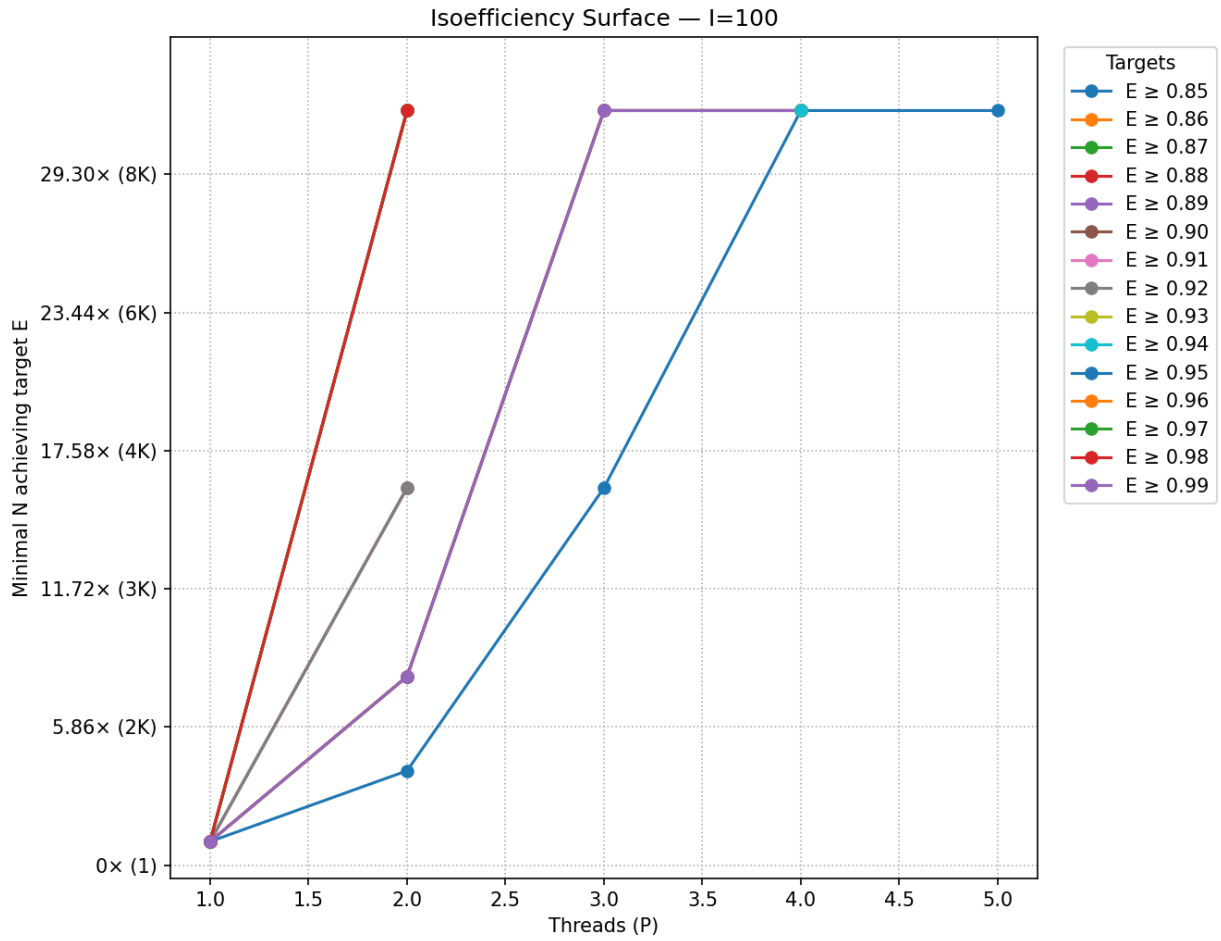
Parallel (using P-threads) 9-Point 2D Stencil Kernel



Parallel (using P-threads) 9-Point 2D Stencil Kernel



Parallel (using P-threads) 9-Point 2D Stencil Kernel



Note – we can't even maintain 85% efficiency at > 5 threads on my laptop – even for plates at the ~ 8K x 8K resolution.

SUBMISSION

Use the same organization as last time, but update to include the new files:

```
(base) wjones@CCU022633 source % tree ./stencil_proj
./stencil_proj
├── code
├── data
├── pres
└── report
```

Parallel (using P-threads) 9-Point 2D Stencil Kernel

Report: **Add to** your LaTex report to include a section in the parallelization, the new code, scripts, discussion of what pthreads are, why we're using them – and then include parallel results from Expanse. Discuss conclusions drawn.

Presentation: Same thing here – take your existing PPT and **add** 4 more minutes to it discussion pthreads, the parallelization, and the data collection and then the results, conclusions drawn from the data on Expanse.

PREVIOUS ASSIGNMENT Sequential Stencil

In this assignment, we're going to create a nine-point stencil C program.

This is the folder structure for your project

```
(base) wjones@CCU022633 source % tree ./stencil_proj
./stencil_proj
├── code
├── data
├── pres
└── report
```

The initial conditions should be that the walls of both left and right sides should be 1.0 and the top and bottom should be 0.0 (see below example of prints)

Create a project, with its own Makefile, that contain the following files:

Makefile
make-2d.c // makes the initial conditions and boundary values
print-2d.c // prints to screen in appealing format, the contents of a data file
stencil-2d.c // performs the stencil operations on data, see below
utilities.h // header containing prototypes for any user functions
utilities.c // source for any user functions (especially any shared between programs)

make-movie.py // make a movie out of prior generated "all iterations" file

run-all.py // invoke all steps defaults are 100x100 x500 iterations – choose some file names

python ./make-movie.py <input .dat file (see below)> <output movie MP4 file>

```
~/D/c/C/S/h/2/source ❯❯❯ ls
Makefile     make-2d.c    print-2d.c   stencil-2d.c utilities.c  utilities.h
~/D/c/C/S/h/2/source ❯❯❯ make
gcc -g -Wall -Wstrict-prototypes -std=gnu99 -c utilities.c
gcc -g -Wall -Wstrict-prototypes -std=gnu99 -c print-2d.c
gcc -lm -o print-2d utilities.o print-2d.o
gcc -g -Wall -Wstrict-prototypes -std=gnu99 -c make-2d.c
gcc -lm -o make-2d utilities.o make-2d.o
gcc -g -Wall -Wstrict-prototypes -std=gnu99 -c stencil-2d.c
gcc -lm -o stencil-2d utilities.o stencil-2d.o
~/D/c/C/S/h/2/source ❯❯❯ ls
Makefile     make-2d.c    print-2d     print-2d.o   stencil-2d.c utilities.c  utilities.o
make-2d      make-2d.o    print-2d.c   stencil-2d   stencil-2d.o utilities.h
~/D/c/C/S/h/2/source ❯❯❯
```

Usages, and showing make-2d and print-2d do their “work”

```
~/D/c/C/S/h/2/source ❯❯❯ ./make-2d
usage: ./make-2d <rows> <cols> <output_file>
~/D/c/C/S/h/2/source ❯❯❯ ./make-2d 5 4 ./initial.dat
~/D/c/C/S/h/2/source ❯❯❯ ./print-2d
./print-2d,
usage: ./print-2d <input data file>
~/D/c/C/S/h/2/source ❯❯❯ ./print-2d ./initial.dat
./print-2d, ./initial.dat,
reading in file: ./initial.dat
1.00    0.00    0.00    1.00
1.00    0.00    0.00    1.00
1.00    0.00    0.00    1.00
1.00    0.00    0.00    1.00
1.00    0.00    0.00    1.00
~/D/c/C/S/h/2/source ❯❯❯
```

Note, initial.dat is a binary file (doubles) stored in row major order, with two pieces of meta data (two ints, representing the number of rows followed by the number of columns, so that we can know the extent of the data to print it, make 2D arrays, etc). As such, if we hex dump the file we can see that:

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
~/D/c/C/S/h/2/source ❯❯❯ hexdump ./initial.dat
0000000 05 00 00 00 04 00 00 00 00 00 00 00 00 00 f0 3f
0000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0000020 00 00 00 00 00 00 f0 3f 00 00 00 00 00 00 f0 3f
0000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0000040 00 00 00 00 00 00 f0 3f 00 00 00 00 00 00 f0 3f
0000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0000060 00 00 00 00 00 00 f0 3f 00 00 00 00 00 00 f0 3f
0000070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0000080 00 00 00 00 00 00 f0 3f 00 00 00 00 00 00 f0 3f
0000090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000a0 00 00 00 00 00 00 f0 3f
00000a8
~/D/c/C/S/h/2/source ❯❯❯ 
```

(note, we're likely using little endian byte order)

In this assignment, I have posted this exact data file (called initial-wjones.dat) in Moodle, and you should diff your file (using Linux command 'diff') to make sure your data file matches mine, since it should be binary equivalent i.e. if your generated file is initial.dat then:

```
~/D/c/C/S/h/2/source ❯❯❯ diff ./initial.dat ./initial-wjones.dat
~/D/c/C/S/h/2/source ❯❯❯ 
```

(diff shows no difference)

Stencil Program: -- How to allocate a 2D array in C. See appendix of this code:

Usage and sample execution for 10 iterations, on the above 5x4 initial.dat file, producing two files:

final.dat // metal plate final end state

all.5x4x10.dat // image stack, like from class, containing data from plate BEFORE every iteration and then also after the last iteration (**so 11 plates worth of data**) Image stack uses the same structure, it has metadata and also then row-major data. Use fread() and fwrite() for binary file IO.

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
~/D/c/C/S/h/2/source ❯❯❯ ./stencil-2d
usage: ./stencil-2d <num iterations> <input file> <output file> <all-iterations>
~/D/c/C/S/h/2/source ❯❯❯ ./stencil-2d 10 ./initial.dat ./final.dat ./all.5x4x10.raw
~/D/c/C/S/h/2/source ❯❯❯
~/D/c/C/S/h/2/source ❯❯❯ ./print-2d ./final.dat
./print-2d, ./final.dat,
reading in file: ./final.dat
1.00    0.00    0.00    1.00
1.00    0.66    0.66    1.00
1.00    0.80    0.80    1.00
1.00    0.66    0.66    1.00
1.00    0.00    0.00    1.00
~/D/c/C/S/h/2/source ❯❯❯
```

In the zip files, I have included these for reference:

```
wjones@CCU022633 files_for_students % ls -lart
total 40
-rw-r--r--   1 wjones  staff   168 Jan 27  2022 initial.dat
-rw-r--r--   1 wjones  staff   168 Jan 27  2022 initial-wjones.dat
-rw-r--r--@  1 wjones  staff   168 Jan 27  2022 final.dat
-rw-r--r--@  1 wjones  staff   818 Jan 27  2022 Makefile
drwxr-xr-x   7 wjones  staff   224 Sep 26 13:06 .
-rw-r--r--   1 wjones  staff  1768 Sep 26 13:13 all.raw.5x4x10.raw
drwxr-xr-x@ 47 wjones  staff  1504 Sep 26 13:13 ..
wjones@CCU022633 files_for_students %
```

See the image stack has a file size of 5 rows x 4 cols x 8 bytes per double x 11 images + 2 ints x 2 bytes per int = 1768 bytes.

This image stack is the intermediate file that you can use as input to a “translate” program that will convert the data into a format that can be used by the python scripts.

stencil-2d will perform a 9-point stencil (not the 5-point) operation:

```
// (NW  + N  + NE +
//   E  + SE + S +
//   SW + W  + C) / 9.0;
newx[i][j] = (x[i-1][j-1] + x[i-1][j]   + x[i-1][j+1] + \
              x[i][j+1]   + x[i+1][j+1] + x[i+1][j] + \
              x[i+1][j-1] + x[i][j-1]   + x[i][j])/9.0;
```

The order of the additions (operations) matters because floating point addition isn’t commutative, so if we want to ensure that all our data matches, this is important (mostly for that reason only here)

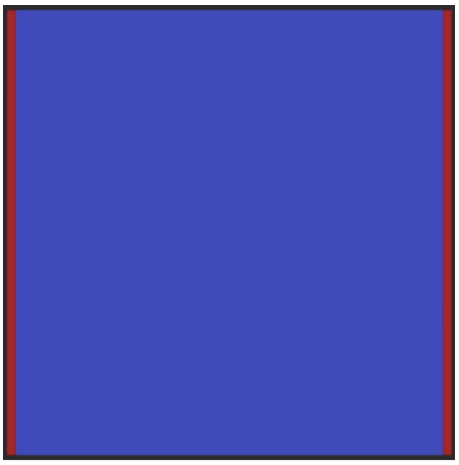
I have included the final.dat and the image stack file from above, so you should make sure you get the exact same files.

Make a python program called display_image.py <inputfile>

That will make a png out if the datafile you put in there – like initial.data – see the borders ?

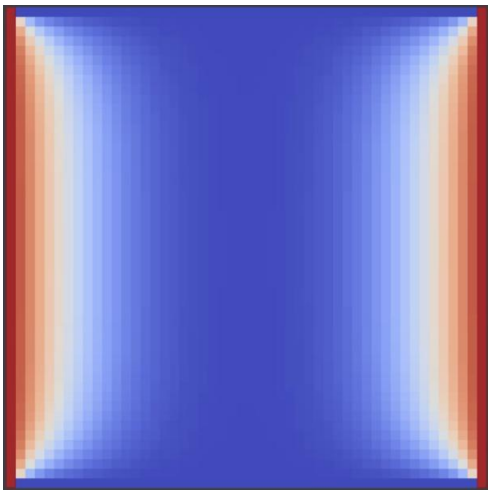
```
python ./display_image.py initial.dat
```

(by default it will write to a png with the same file name (e.g. initial.png)



50 x 50 input plate

After some number of iterations (>10) ::

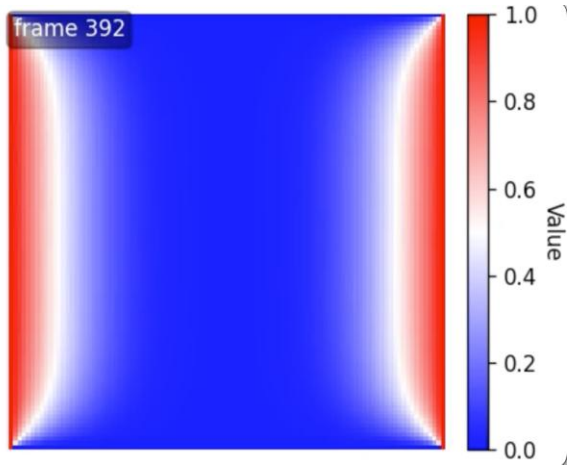


As per the assignment – make sure your python scripts do everything to visualize the data

Parallel (using P-threads) 9-Point 2D Stencil Kernel

Make another python program that makes a movie out of the stack datafile (this one was a 100x100 for 500 iterations (i.e. 501 frames). Here is a screenshot from mine – the movie is also in the assignment to look at.

```
python make_movie.py \
  --in all.100x100x500.dat \
  --out all.100x100x500.mp4
```



Make a LaTex document what your assignment. Just like last time, I want the PDF itself, and also the entire ZIP from Overleaf in the report folder.

Make a 5-minute PPTX with voiceover and the visualization and a movie inside of a 30 x 200 x 1000 iteration.

Submission to Moodle.

Tar GZ the entire project directory (while being in the parent folder of the folder you're trying to compress) up using:

tar -czvf your_file_name.tar.gz ./the_folder_to_compress

Use a file / folder structure like what I've asked for above.

Submit to moodle.

NOTES: Possible Makefile: (you may want to make certain changes)

```
CC=gcc
CFLAGS=-g -Wall -Wstrict-prototypes -std=gnu99
LFLAGS=-lm
PROGS=print-2d make-2d stencil-2d
all:  $(PROGS)
print-2d:   utilities.o print-2d.o
   $(CC) $(LFLAGS) -o print-2d utilities.o print-2d.o
make-2d: utilities.o make-2d.o
   $(CC) $(LFLAGS) -o make-2d utilities.o make-2d.o
stencil-2d: utilities.o stencil-2d.o
   $(CC) $(LFLAGS) -o stencil-2d utilities.o stencil-2d.o
utilities.o: utilities.c utilities.h
   $(CC) $(CFLAGS) -c utilities.c
print-2d.o: print-2d.c utilities.h
   $(CC) $(CFLAGS) -c print-2d.c
make-2d.o: make-2d.c utilities.h
   $(CC) $(CFLAGS) -c make-2d.c
stencil-2d.o: stencil-2d.c utilities.h
   $(CC) $(CFLAGS) -c stencil-2d.c
clean:
   rm -f *.o core* $(PROGS)
```

2D array allocation in C – lots of ways. This is one way:

```
void malloc2D(double***a , int jmax, int imax)
{
   // first allocate a block of memory for the row pointers and the 2D array
   double **x = (double **)malloc(jmax*sizeof(double *) + jmax*imax*sizeof(double));

   // Now assign the start of the block of memory for the 2D array after the row pointers
   x[0] = (double *)x + jmax;

   // Last, assign the memory location to point to for each row pointer
   for (int j = 1; j < jmax; j++) {
      x[j] = x[j-1] + imax;
   }

    *a = x;
}
```

How to call and use that 2D space:

Parallel (using P-threads) 9-Point 2D Stencil Kernel

```
// example 2D array
rows = atoi(argv[1]);
cols = atoi(argv[2]);
double **A;
malloc2D(&A, rows, cols);
fill2D(A, rows, cols);
print2D(A, rows, cols);
```

// note, this does not fill the array with the correct date for this stencil assignment.

Here is some code you can use to do timing – in fact, use this. Place this in a file called "timer.h" and #include it where needed.

```
/* File:     timer.h
 *
 * Purpose:  Define a macro that returns the number of seconds that
 *           have elapsed since some point in the past.  The timer
 *           should return times with microsecond accuracy.
 *
 * Note:     The argument passed to the GET_TIME macro should be
 *           a double, *not* a pointer to a double.
 *
 * Example:
 *    #include "timer.h"
 *    . . .
 *    double start, finish, elapsed;
 *    . . .
 *    GET_TIME(start);
 *    . . .
 *    Code to be timed
 *    . . .
 *    GET_TIME(finish);
 *    elapsed = finish - start;
 *    printf("The code to be timed took %e seconds\n", elapsed);
 *
 * IPP:  Section 3.6.1 (pp. 121 and ff.) and Section 6.1.2 (pp. 273 and ff.)
 */
#ifndef _TIMER_H_
#define _TIMER_H_

#include <sys/time.h>

/* The argument now should be a double (not a pointer to a double) */
#define GET_TIME(now) { \
   struct timeval t; \
   gettimeofday(&t, NULL); \
   now = t.tv_sec + t.tv_usec/1000000.0; \
}

#endif
```